Tipos Abstractos de Datos (TAD)

1. TAD: Lista Enlazada (LinkedList)

Nombre

Lista Enlazada

Descripción

Estructura de datos lineal dinámica compuesta por nodos, donde cada nodo contiene un dato y una referencia al siguiente nodo. Permite almacenar elementos de manera secuencial sin necesidad de un tamaño fijo

Invariante

- La lista puede estar vacía (head = null)
- Cada nodo apunta al siguiente nodo o a null si es el último
- No existen ciclos en la lista
- El tamaño de la lista corresponde al número de nodos enlazados

Operaciones

Operación	Entrada	Salida	Descripción
add(element)	Elemento	void	Agrega un elemento al final de la lista
remove(element)	Elemento	boolean	Elimina la primera ocurrencia del elemento
get(index)	int	Elemento	Retorna el elemento en la posición dada
getLength()	-	int	Retorna el tamaño de la lista
isEmpty()	-	boolean	Indica si la lista está vacía
iterator()	-	Iterator	Permite recorrer la lista

2. TAD: Árbol Binario de Búsqueda (BinarySearchTree)

Nombre

Árbol Binario de Búsqueda

Descripción

Estructura jerárquica en la cual cada nodo contiene un dato y dos referencias: hijo izquierdo y derecho. Los elementos se organizan de forma ordenada, donde los valores menores van a la izquierda y los mayores a la derecha

Invariante

- Para cada nodo:
    - Todos los elementos del subárbol izquierdo son menores
    - Todos los elementos del subárbol derecho son mayores
- No hay elementos duplicados
- Existe un único nodo raíz

Operaciones

Operación	Entrada	Salida	Descripción
add(element)	Elemento	void	Inserta un elemento en el árbol
search(element)	Elemento	Nodo	Busca un elemento en el árbol
delete(element)	Elemento	void	Elimina un elemento del árbol
isEmpty()	-	boolean	Indica si el árbol está vacío
getRoot()	-	Nodo	Retorna la raíz del árbol

3. TAD: Pila (Stack)

Nombre

Pila

Descripción

Estructura de datos lineal que sigue el principio LIFO (Last In, First Out), donde el último elemento en entrar es el primero en salir

Invariante

- Solo se puede acceder al elemento en la cima
- El tamaño corresponde al número de elementos almacenados
- No se permite acceso directo a elementos intermedios

Operaciones

Operación	Entrada	Salida	Descripción
push(element)	Elemento	void	Inserta un elemento en la cima
pop()	-	Elemento	Elimina y retorna el elemento superior
peek()	-	Elemento	Retorna el elemento superior sin eliminarlo
isEmpty()	-	boolean	Indica si la pila está vacía

4. TAD: Cola (Queue)

Nombre

Cola

Descripción

Estructura de datos lineal que sigue el principio FIFO (First In, First Out), donde el primer elemento en entrar es el primero en salir

Invariante

- Inserción por el final y eliminación por el inicio
- El orden de los elementos se mantiene
- No se accede directamente a posiciones intermedias

Operaciones

Operación	Entrada	Salida	Descripción
enqueue(element)	Elemento	void	Inserta un elemento al final
dequeue()	-	Elemento	Elimina y retorna el primer elemento
peek()	-	Elemento	Retorna el primer elemento sin eliminarlo
isEmpty()	-	boolean	Indica si la cola está vacía

5. TAD: GameState

Nombre

Estado del Juego

Descripción

Representa el estado actual del juego Sokoban, incluyendo el tablero, la posición del jugador y el nivel actual

Invariante

- El tablero no es nulo
- La posición del jugador es válida dentro del tablero
- El estado representa una configuración válida del juego

Operaciones

Operación	Entrada	Salida	Descripción
getBoard()	-	Board	Retorna el tablero
getPlayerPosition()	-	Position	Retorna la posición del jugador
setPlayerPosition(pos)	Position	void	Actualiza la posición del jugador
getMoveCount()	-	int	Retorna número de movimientos
getPushCount()	-	int	Retorna número de empujes